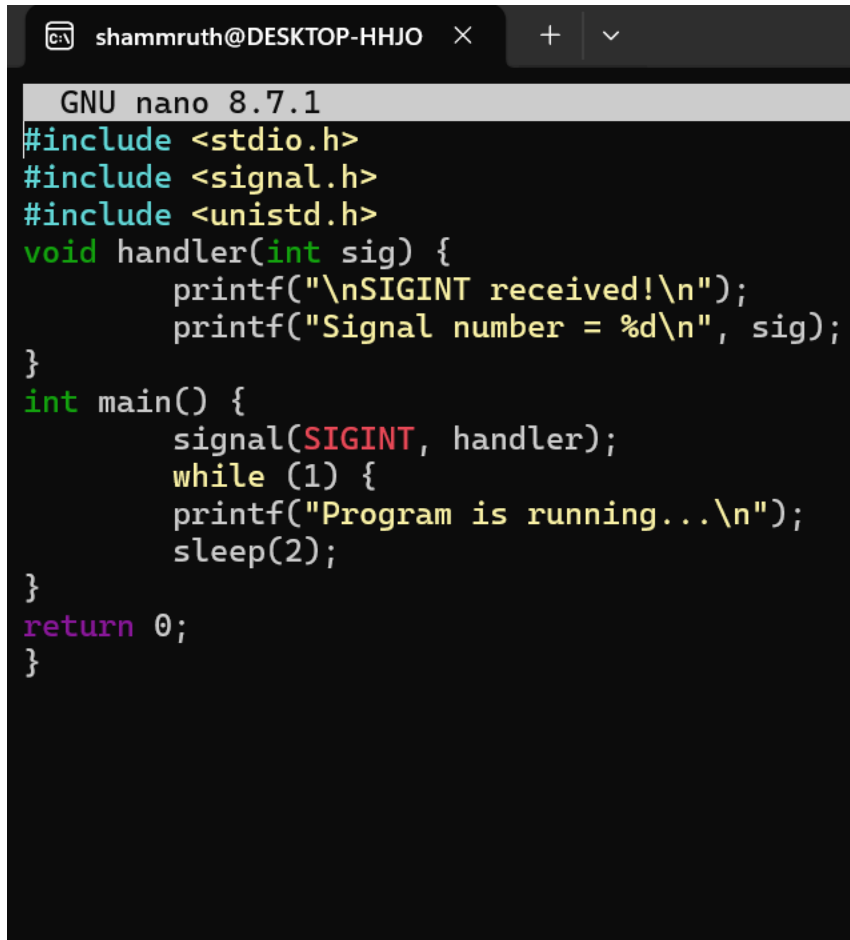


Signals



The image shows a terminal window with a dark background. At the top, the terminal title bar displays 'shammruth@DESKTOP-HHJO' followed by window control icons. Below the title bar, the prompt 'GNU nano 8.7.1' is visible. The main area contains C code for a signal handler. The code includes headers for stdio, signal, and unistd. It defines a handler function for SIGINT that prints a message and the signal number. The main function sets up the handler and enters a loop that prints 'Program is running...' and sleeps for 2 seconds.

```
GNU nano 8.7.1
#include <stdio.h>
#include <signal.h>
#include <unistd.h>
void handler(int sig) {
    printf("\nSIGINT received!\n");
    printf("Signal number = %d\n", sig);
}
int main() {
    signal(SIGINT, handler);
    while (1) {
        printf("Program is running...\n");
        sleep(2);
    }
    return 0;
}
```

Out put

```
P4      4      12      16
shamruth@DESKTOP-HHJODKP:~$ nano signal.c
shamruth@DESKTOP-HHJODKP:~$ gcc signal.c -o signal
shamruth@DESKTOP-HHJODKP:~$ ./signal
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
^C
SIGINT received!
Signal number = 2
Program is running...
Program is running...
Program is running...
^Z
[1]+  Stopped                  ./signal
shamruth@DESKTOP-HHJODKP:~$ |
```

SIGTERM

[illegible]

SIGCHLD

```
shamruth@DESKTOP-HHJO  ×  +  ▾
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

void handle_sigchld(int sig)
{
    printf("\nSIGCHLD received: Child process finished.\n");
    wait(NULL);
}

int main()
{
    pid_t pid;

    signal(SIGCHLD, handle_sigchld);

    pid = fork();

    if (pid < 0)
    {
        printf("Fork failed.\n");
        return 1;
    }

    if (pid == 0)
    {
        printf("Child process started.\n");
        sleep(3);
        printf("Child process exiting.\n");
        return 0;
    }
    else
    {
        printf("Parent process started.\n");
        printf("Parent waiting for child...\n");

        while (1)
        {
            sleep(1);
        }
    }

    return 0;
}
```

Out put

```
shammruth@DESKTOP-HHJODKP:~$ nano sigchild.c
shammruth@DESKTOP-HHJODKP:~$ gcc sigchild.c -o sigchild
shammruth@DESKTOP-HHJODKP:~$ ./sigchild
Parent process started.
Parent waiting for child...
Child process started.
Child process exiting.

SIGCHLD received: Child process finished.
```

SIGPIPE

```
shammruth@DESKTOP-HHJODKP:~$ nano sigpipe.c
shammruth@DESKTOP-HHJODKP:~$ gcc sigpipe.c -o sigpipe
shammruth@DESKTOP-HHJODKP:~$ ./sigpipe
Reading end of pipe closed.
Writing to pipe...
SIGPIPE received.
Program finished.
shammruth@DESKTOP-HHJODKP:~$
```

SIGSEGV

```
shamruth@DESKTOP-HHJO  ×  +  v
GNU nano 8.7.1
#include <stdio.h>
#include <signal.h>
#include <stdlib.h>

void handle_sigsegv(int sig)
{
    printf("\nSIGSEGV received.\n");
    printf("Invalid memory access detected.\n");

    exit(1);
}

int main()
{
    int *ptr = NULL;

    signal(SIGSEGV, handle_sigsegv);

    printf("Program started.\n");
    printf("Trying to access invalid memory..");

    *ptr = 10;

    printf("This line will not execute.\n");

    return 0;
}
```

Out put

```
shamruth@DESKTOP-HHJODKP:~$ nano sigsegv.c
shamruth@DESKTOP-HHJODKP:~$ gcc sigsegv.c -o sigsegv
shamruth@DESKTOP-HHJODKP:~$ ./sigsegv
Program started.
Trying to access invalid memory...

SIGSEGV received.
Invalid memory access detected.
shamruth@DESKTOP-HHJODKP:~$ |
```